

Transformer Interview Questions and Solutions

Transformer Fundamentals

1. What problem did the Transformer solve compared with RNNs/LSTMs?

Answer:

RNNs process tokens sequentially, which makes training slow and makes long-range dependency modeling difficult. Transformers replace recurrence with self-attention, allowing all tokens in a sequence to attend to each other in parallel.

Key benefits:

- Parallel training
- Better long-range dependency modeling
- Scales well with data and model size
- Foundation of modern LLMs such as GPT, BERT, T5, etc.

2. What are the main components of a Transformer block?

Answer:

A standard Transformer block usually contains:

1. Multi-head self-attention
2. Feed-forward neural network
3. Residual connections
4. Layer normalization
5. Dropout during training

For decoder-only models like GPT:

Input

- > LayerNorm
- > Masked Multi-Head Self-Attention
- > Residual Add
- > LayerNorm
- > Feed-Forward Network
- > Residual Add
- > Output

3. What is self-attention?

Answer:

Self-attention allows each token in a sequence to compute a weighted combination of all other tokens.

Each token produces:

- Query, Q
- Key, K
- Value, V

Formula:

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$

4. Why divide by $\sqrt{d_k}$?

Answer:

Without scaling, dot products between Q and K can become large when the key/query dimension is large. Large values make softmax very sharp, causing small gradients and unstable training. Dividing by $\sqrt{d_k}$ keeps attention scores numerically stable.

5. What is multi-head attention?

Answer:

Multi-head attention runs several attention operations in parallel. Each head can learn different relationships between tokens, such as syntax, locality, or long-range dependencies.

6. Why not use just one attention head?

Answer:

A single head may capture only one type of relationship. Multiple heads allow the model to attend to different representation subspaces. However, some heads can become redundant.

Positional Encoding

7. Why does a Transformer need positional encoding?

Answer:

Self-attention is permutation-invariant by itself. It does not naturally know token order. Positional encoding injects order information into the model.

8. What are sinusoidal positional encodings?

Answer:

The original Transformer used fixed sine and cosine functions of different frequencies to represent token positions. They can generalize better to unseen sequence lengths than learned absolute embeddings in some settings.

9. What are learned positional embeddings?

Answer:

Learned positional embeddings assign each position a trainable vector. They are simple and effective, but often tied to the maximum sequence length used during training.

10. Absolute vs relative positional encoding?

Answer:

Absolute encoding gives the exact token position. Relative encoding tells the model how far apart two tokens are. Relative encodings often help with longer contexts and local structure.

Encoder, Decoder, and Architectures

11. Difference between Transformer encoder and decoder?

Answer:

The encoder uses bidirectional self-attention and can attend to the full input. The decoder uses causal masked self-attention so each token only attends to previous tokens.

12. What is causal masking?

Answer:

Causal masking prevents a token from attending to future tokens. This is required for autoregressive generation.

13. What is cross-attention?

Answer:

Cross-attention is used in encoder-decoder models. Queries come from the decoder, while keys and values come from the encoder.

14. Compare BERT, GPT, and T5.

Model	Architecture	Attention Type	Typical Use
BERT	Encoder-only	Bidirectional	Classification, extraction, embeddings
GPT	Decoder-only	Causal	Text generation
T5	Encoder-decoder	Encoder bidirectional, decoder causal	Text-to-text tasks

15. Why is BERT not naturally suited for left-to-right generation?

Answer:

BERT is trained with masked language modeling and bidirectional attention, not next-token prediction. GPT is trained autoregressively, making it better suited for generation.

Training Objectives

16. What is masked language modeling?

Answer:

Masked language modeling randomly masks input tokens and trains the model to predict them using surrounding context.

Example:

The cat sat on the [MASK].

17. What is autoregressive language modeling?

Answer:

Autoregressive modeling predicts the next token from previous tokens.

$$P(x_1, \dots, x_n) = P(x_1)P(x_2|x_1)\dots P(x_n|x_1\dots x_{n-1})$$

18. What is teacher forcing?

Answer:

Teacher forcing gives the model the ground-truth previous token during training. During inference, the model uses its own previously generated token.

19. What is exposure bias?

Answer:

Exposure bias occurs because training uses correct previous tokens, while inference uses the model's own predictions. Early mistakes can compound.

Feed-Forward Networks and Normalization

20. What is the feed-forward network in a Transformer?

Answer:

It is a position-wise MLP applied independently to each token:

$$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 x + b_1) + b_2$$

Usually d_{ff} is larger than d_{model} , often around $4 \times d_{\text{model}}$.

21. Why use residual connections?

Answer:

Residual connections improve gradient flow and allow layers to learn refinements rather than entirely new representations.

22. What is layer normalization?

Answer:

Layer normalization normalizes activations across the feature dimension for each token. It stabilizes training and works well for sequence models.

23. Pre-LN vs Post-LN?

Answer:

Post-LN applies LayerNorm after the residual addition. Pre-LN applies LayerNorm before the sublayer. Modern large Transformers often use Pre-LN because it improves training stability.

Complexity and Efficiency

24. What is self-attention complexity?

Answer:

For sequence length n and hidden size d :

Time complexity: $O(n^2 d)$
Memory complexity: $O(n^2)$

25. Why is long-context attention expensive?

Answer:

Attention compares every token with every other token. Doubling the sequence length roughly quadruples the attention matrix size.

26. How can attention be made more efficient?

Answer:

Common approaches include sparse attention, sliding-window attention, linear attention, FlashAttention, multi-query attention, grouped-query attention, and KV caching.

27. What is FlashAttention?

Answer:

FlashAttention computes exact attention more memory-efficiently by using blockwise computation and avoiding materializing the full attention matrix in high-bandwidth memory.

28. What is KV caching?

Answer:

KV caching stores past keys and values during autoregressive inference so the model does not recompute them for every new token.

29. What is multi-query attention?

Answer:

Multi-query attention lets multiple query heads share the same key and value projections. This reduces KV-cache memory and speeds up inference.

30. What is grouped-query attention?

Answer:

Grouped-query attention is a compromise where groups of query heads share key-value projections.

Implementation and Practical Questions

31. Shapes of Q, K, and V?

Given:

B = batch size
T = sequence length
D = model dimension
H = number of heads
 $D_h = D / H$

Shapes:

Input X: [B, T, D]
Q, K, V: [B, H, T, D_h]
Attention scores: [B, H, T, T]
Output: [B, T, D]

32. How do you implement causal masking?

Answer:

Use a lower-triangular matrix so each token can only attend to itself and previous tokens.

33. Why softmax over attention scores?

Answer:

Softmax converts scores into positive weights that sum to 1 over the key dimension.

34. What happens without positional encoding?

Answer:

The model loses explicit token order information, hurting sequence-sensitive tasks.

35. What is dropout used for?

Answer:

Dropout regularizes embeddings, attention weights, feed-forward layers, and residual paths.

Advanced Questions

36. Why are Transformers easier to parallelize than RNNs?

Answer:

RNNs process tokens sequentially. Transformers process all tokens in parallel during training using self-attention.

37. What is attention collapse?

Answer:

Attention collapse occurs when attention becomes overly concentrated on a few tokens or patterns, reducing context usage.

38. Are attention weights explanations?

Answer:

Not always. Attention weights can be useful diagnostics but are not guaranteed to faithfully explain model reasoning.

39. Why do LLMs often use decoder-only Transformers?

Answer:

Decoder-only Transformers are simple, scalable, and effective for next-token prediction, which can support many language tasks.

40. Training vs inference in GPT?

Answer:

Training processes all tokens in parallel with causal masking. Inference generates one token at a time, usually using KV caching.

Senior-Level Topics

41. What is RoPE?

Answer:

Rotary positional embedding encodes position by rotating query and key vectors. It naturally incorporates relative position information into attention.

42. What is ALiBi?

Answer:

ALiBi adds distance-based linear biases to attention scores, penalizing farther tokens and helping length extrapolation.

43. What is RMSNorm?

Answer:

RMSNorm normalizes by root mean square without subtracting the mean. It is simpler than LayerNorm and common in modern LLMs.

44. What is SwiGLU?

Answer:

SwiGLU is a gated feed-forward activation often used in modern Transformers for improved expressiveness.

45. What is mixture-of-experts?

Answer:

MoE uses multiple expert feed-forward networks and routes each token to a subset of experts, increasing capacity without activating all parameters per token.

Scenario-Based Questions

46. Your Transformer is running out of GPU memory. What can you do?

Answer:

Reduce batch size or sequence length, use gradient checkpointing, mixed precision, FlashAttention, model parallelism, smaller model dimensions, or parameter-efficient fine-tuning.

47. Your model generates repetitive text. What could be wrong?

Answer:

Causes include greedy decoding, low temperature, poor data quality, overfitting, or lack of repetition penalties. Fixes include top-p sampling, higher temperature, and better data.

48. Transformer performs poorly on long documents. Why?

Answer:

Possible causes include limited context window, poor positional extrapolation, high attention cost, and lack of long-context training data.

49. Why might more attention heads not improve performance?

Answer:

With fixed `d_model`, more heads mean smaller head dimension. Some heads may also become redundant.

50. Explain a Transformer to a non-technical person.

Answer:

A Transformer reads text by letting every word look at other words and decide which ones matter most for understanding the sentence.